

CSCI-UA 102 — Quiz 3

Recursion

Name: _____ NetID: _____

funNum

For an integer array of size N , I define the **funNum** of the array as follows:

- If the array has **no elements**, then funNum is **0**.
- If the array has **one element**, then funNum is equal to **that element**.
- Otherwise, funNum is equal to the **sum** of:
 1. the last element of the array,
 2. the funNum of the first $N - 1$ elements, and
 3. the funNum of the first $N - 2$ elements.

Examples:

- $\text{funNum}(\{\}) = 0$
- $\text{funNum}(\{5\}) = 5$
- $\text{funNum}(\{5, 3\}) = 3 + 5 + 0 = 8$
- $\text{funNum}(\{5, 3, 1\}) = 1 + 8 + 5 = 14$

Write an efficient, recursive method:

```
public int funNum(int[] array)
```

What is the time complexity of your solution? _____

Solution

Key insight: `funNum` only depends on the prefix length, so the recurrence is $f(n) = \text{array}[n-1] + f(n-1) + f(n-2)$, identical in structure to Fibonacci. Naive recursion is $O(2^n)$; an efficient solution avoids recomputation.

Efficient solution — **return previous values** (same pattern as Fibonacci):

```
public int funNum(int[] array) {
    if (array.length == 0) return 0;
    if (array.length == 1) return array[0];
    return funNumPair(array, array.length)[0];
}

// returns {funNum(first n elems), funNum(first n-1 elems)}
private int[] funNumPair(int[] array, int n) {
    if (n == 1) return new int[]{array[0], 0};
    int[] prevPair = funNumPair(array, n - 1);
    // prevPair[0] = f(n-1), prevPair[1] = f(n-2)
    int funNumN = array[n - 1] + prevPair[0] + prevPair[1];
    return new int[]{funNumN, prevPair[0]};
}
```

The `funNumPair` method recurses naturally on $n-1$ but returns *two* values: $f(n)$ and $f(n-1)$, so the caller gets both values it needs from a single recursive call instead of two, eliminating the branching.

Time complexity: $O(n)$ — each prefix length is computed exactly once.